The Journal of Open Source Software

Piecewise: Flexible piecewise functions for fast integral transforms in Julia

Christophe Berthod [1]

1 Department of Quantum Matter Physics, University of Geneva, 1205 Geneva, Switzerland

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: George K. Thiruvathukal

Reviewers:

- @ranocha
- @dawbarton

Submitted: 04 April 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

A piecewise function of a real variable x returns a value computed from a rule that can be different in each interval of the values of x. The Julia (Bezanson et al., 2017) module `Piecewise` provides an implementation of piecewise functions, where the user is free to choose the rules. A mechanism allows for fitting a piecewise function made of user-defined formulas to a real function of a real variable. With appropriately chosen formulas, various integral transforms of the piecewise function become directly available without relying on quadratures. The module `Piecewise` defines seven formula that enable the fast calculation of the moments of the piecewise function. The module `PiecewiseHilbert` supplements these formula with methods enabling a fast Hilbert transform. The module `PiecewiseLorentz` extends some of these formula to enable what we call a Lorentz transform. This code was written to solve a quantum physics problem involving several coupled integral equations (Berthod, 2025; Morpurgo et al., 2024, 2025).

Statement of need

The interpolation problem, which consists in constructing a continuous function out of discrete data, is ubiquitous in many areas of science and technology. This problem has been traditionally solved by means of global or piecewise polynomial functions (Bhagavan et al., 2024; Kittisopikul et al., 2023; Wikibooks, 2021). The various interpolation schemes differ by the order of the polynomials, the choice of the sampling points when this choice is possible, and the additional conditions required when the solution is not uniquely determined by the data. Beside drawing a smooth curve through points in a graph, one important use of interpolations is for constructing a cheap but accurate approximation of a computer-intensive function. If the latter function presents critical points like discontinuities or singularities, all polynomial interpolations fail in the neighborhood of these points, due to the absence of a convergent Taylor series. When the underlying function has critical points and accuracy is an issue, there is a need for piecewise interpolation schemes that are based on nonanalytic functions rather than polynomials.

The mathematical problems involving integral equations (i.e., when the unknown function appears inside an integral) are often solved numerically by discretizing the integral and setting up an iteration. This introduces a discretization error. A choice of the discrete grid that minimizes the error would generally be nonuniform and require *a priori* knowledge of the solution. An optimization of the grid is possible through iterative refinement. However, if the actual solution has critical points, the iterative refinement will likely fail. Another approach is to represent the solution at iteration n as a piecewise function, evaluate the integrals using quadratures, and fit a new piecewise function to the solution computed at iteration $n+1$. This algorithm may not be faster than the discretization approach, but it eliminates the discretization bias. Furthermore, the critical points can in principle be captured in a piecewise function involving appropriate nonanalytic functions. The procedure requires recursively fitting

a set of elementary functions, including nonanalytic ones that are problem dependent, to a given function, until a sufficient accuracy is achieved in each piece. To our knowledge, no Julia package offers this functionality.

A subclass of all integral equations comprises those involving linear integral transforms of the kind $(K \circ f)(\mathbf{X}) = \int_{-\infty}^{\infty} dx\, f(x) K(x, \mathbf{X})$, where $f(x)$ is a function of a real variable x and $K(x, \mathbf{X})$ is a kernel depending on x and another, possibly multidimensional, variable $\mathbf{X}$. For instance, the n-th moment of a distribution is the integral transform of this distribution with kernel $K(x, n) = x^n$. Other examples include the Fourier transform with $K(x, k) = e^{-ikx}$, the Laplace transform with $K(x, s) = \theta(x) e^{-sx}$, $\theta(x)$ being the Heaviside step function, the Hilbert transform with $K(x, y) = 1/(y - x + i0^+)$, or more generally $K(x, z) = 1/(z - x)$ with $z \in \mathbb{C} \setminus \mathbb{R}$ a complex number with finite imaginary part. If the function $f(x)$ is represented as a piecewise function, and if the various elementary functions $F_i(x)$ used in this piecewise representation are such, that the solution of the differential equation $\frac{d}{dx} P_i(x, \mathbf{X}) = F_i(x) K(x, \mathbf{X})$ is known exactly, then $(K \circ f)(\mathbf{X})$ is immediately available by evaluating the functions $P_i(x, \mathbf{X})$ at the boundaries of each piece. This may significantly outperform the evaluation of $(K \circ f)(\mathbf{X})$ using quadratures, especially near the critical points of $f(x)$, where the quadratures typically converge slowly, if they converge at all. Thus, an environment where problem-dependent functions $F_i(x)$ may be defined and used in piecewise functions, together with kernel-dependent functions $P_i(x, \mathbf{X})$, is desirable.

The `Piecewise` modules

The module `Piecewise` provides such an environment based on three structures. A structure called `Formula` holds a user-defined function depending on a given number of parameters, together with possible restrictions regarding the values of these parameters with respect to the interval in which the formula is used. A second structure called `Piece` holds an interval, a rule that can be a sum of `Formula` objects, and the parameters to be passed to these formula. Finally, a structure called `PiecewiseFunction` holds a collection of `Piece` objects. The module comes with a small set of pre-defined `Formula` that should cover a wide variety of cases and a method for fitting a `PiecewiseFunction` object to a given function. Note that, unlike polynomial interpolations, this fitting does not enforce *exact* continuity at the boundaries of the pieces. The pre-defined `Formula` earn additional methods in the module `PiecewiseHilbert`, such that the Hilbert transform of piecewise functions using these formula can be evaluated without using quadratures. The module `PiecewiseLorentz` offers this functionality for another integral transform (see the documentation).

Examples

1. Two tutorials are available as Jupyter notebooks:
   - Tutorial 1: Constructing approximations with `Piecewise.piecewisefit`
   - Tutorial 2: Solving an implicit equation using `PiecewiseHilbert`
2. For a complete use case, see `MagnetoTransport.jl`.
3. For an example of nonlinear integral equation solved using `Piecewise`, see van der Marel & Berthod (2024).
4. A simple demonstration is provided below.

Electronic density of states (DOS) functions typically have critical points. The DOS is derived from a dispersion relation $\varepsilon(\mathbf{k})$ as $N(E) = \int \frac{\mathrm{d}^d k}{(2\pi)^d} \delta(E - \varepsilon(\mathbf{k}))$ in dimension d, where $\delta(\cdot)$ is the Dirac delta function. $N(E)$ has critical points whenever $\nabla \varepsilon(\mathbf{k}) = 0$ for some $\mathbf{k}$. The DOS is an ingredient of many calculations, but in general it is not known exactly. In the following

illustration, we construct a one-piece approximation with relative accuracy below 10^{-5} for such a DOS function.

Electrons hopping with unit energy between neighboring sites of a two-dimensional square lattice with unit lattice parameter have a dispersion relation $\varepsilon(\mathbf{k}) = 2(\cos k_x + \cos k_y)$. This case is peculiar in that the DOS is known exactly: $N(E) = K(1-(E/4)^2)\theta(4-|E|)/(2\pi^2)$ with K the elliptic function. It has two discontinuities at $E = \pm 4$ and a logarithmic singularity at $E = 0$. See Tutorial 1 for further details.

```julia
using SpecialFunctions: ellipk
using Piecewise

# DOS function. Due to the dependence on E^2, ellipk(1 - (E / 4)^2)
# looses accuracy for ¦E¦ < 1e-4. We use the known expansion instead.
N(E) = (abs(E) < 1e-4 ? log(16 / abs(E)) : abs(E) > 4 ? 0.0 :
    ellipk(1 - (E / 4)^2)) / (2 * π^2)

# Piecewise function representing the logarithmic singularity
singularity = PiecewiseFunction(:even,
    Piece((0, 4), (false, true), LOG, [0, -1 / (2 * π^2)]))

# It is better to remove the singularity for fitting and to add it
# afterwards. PiecewiseFunction objects can be summed.
f = piecewisefit(E -> N(E) - singularity(E),
    (0, 4), [POLY], parity=:even, rtol=5e-6)
f += singularity
```

```
< Piecewise even function with 1 piece and support [-4.0, 4.0] >
```

```julia
# Check that the relative error is smaller than 1e-5
maximum(LinRange(-4, 4, 1000) .|> E -> abs(f(E) ./ N(E) - 1)) < 1e-5
```

```
true
```

```julia
# Printing a PiecewiseFunction shows the constructor for that object.
# The exact numbers may vary, as randomness is involved in the fitting.
println(f)
```

```
PiecewiseFunction(:even, [
    Piece((0.0, 4.0), (false, true), [POLY, LOG],
        [[1.404609620501190e-01, 1.174637035445451e-04, 2.462481494536943e-03,
        -1.995071066151247e-03, 1.349285972889321e-03, -6.607932327688245e-04,
        2.158678002039584e-04, -4.417944490515085e-05, 5.103263826300048e-06,
        -2.533300199959029e-07], [0.000000000000000e+00,
        -5.066059182116889e-02]])
])
```

References

Berthod, C. (2025). *Linear magneto-transport with a local self-energy in Julia*. https://github.com/ChristopheBerthod/MagnetoTransport.jl.

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65. https://doi.org/10.1137/141000671

Bhagavan, S., de Koning, B., Maddhashiya, S., & Rackauckas, C. (2024). DataInterpolations.jl: Fast interpolations of 1D data. *Journal of Open Source Software*, *9*(101), 6917. https://doi.org/10.21105/joss.06917

Kittisopikul, M., Holy, T. E., & Aschan, T. (2023). *JuliaMath/Interpolations.jl: v0.15.1*. https://github.com/JuliaMath/Interpolations.jl.

Morpurgo, G., Berthod, C., & Giamarchi, T. (2025). Universal low-density power laws of the dc conductivity and Hall constant in the self-consistent Born approximation. *arXiv:2504.02739*. https://doi.org/10.48550/arXiv.2504.02739

Morpurgo, G., Rademaker, L., Berthod, C., & Giamarchi, T. (2024). Hall response of locally correlated two-dimensional electrons at low density. *Physical Review Research*, *6*, 013112. https://doi.org/10.1103/PhysRevResearch.6.013112

van der Marel, D., & Berthod, C. (2024). Superconductivity in metallic hydrogen. *Newton*, *1*, 100002. https://doi.org/10.1016/j.newton.2024.100002

Wikibooks. (2021). *Introduction to numerical methods — Wikibooks, the free textbook project*. https://en.wikibooks.org/wiki/Introduction_to_Numerical_Methods/Interpolation

DRAFT